

Implementation and Experimental Analysis of the Guruswami-Sudan List Decoder using SageMath

Nadav Cremisi, Ori Kremer
Error Correcting Codes

February 15, 2026

Abstract

This report presents a complete implementation of the Guruswami-Sudan (GS) list decoding algorithm for Reed-Solomon codes. Using SageMath for the underlying algebraic operations, we demonstrate the algorithm's ability to correct errors beyond the unique decoding bound of $(n - k)/2$. We conduct a series of experiments to analyze the trade-offs between the multiplicity parameter m , runtime complexity, and error-correction radius. Our results confirm that while the GS algorithm achieves the theoretical Johnson radius, practical implementation requires balancing the multiplicity parameter against computational cost.

1 Introduction

Reed-Solomon (RS) codes are the standard for error correction in storage and communication systems due to their optimality regarding the singleton bound. However, traditional unique decoding algorithms (such as Berlekamp-Massey) are fundamentally limited by the *unique decoding radius* of $\lfloor (n - k)/2 \rfloor$. When the error count surpasses this bound, unique decoders fail, discarding potentially recoverable data. The challenge, therefore, is to recover messages from corrupted codewords even when the noise level exceeds half the minimum distance of the code. The primary objective of this project is to implement and analyze the Guruswami-Sudan (GS) List Decoding algorithm, which theoretically extends the error-correction capability up to the Johnson bound. Specific goals include:

1. **Implementation:** Develop a functional GS decoder in SageMath capable of varying the multiplicity parameter m to trade off complexity for error tolerance.
2. **Validation:** Empirically verify that the decoder can correct errors beyond the unique bound and approach the Johnson bound.
3. **Performance Analysis:** Quantify the computational cost (runtime) of the interpolation step and determine the practical limits of increasing m .
4. **Behavioral Study:** Investigate the nature of "list" outputs—specifically, whether multiple candidates appear under random noise versus adversarial signal collisions.

We present a complete implementation of the Guruswami-Sudan algorithm. Our solution utilizes bivariate polynomial interpolation to construct a curve $Q(x, y)$ that passes through the received points with multiplicity m , followed by a root-finding step to extract valid message polynomials $y - P(x)$.

2 Implementation Details

2.1 Initialization

```
1 def __init__(self, field, n, k, m=None):
2     self.F = field
3     self.n = n
4     self.k = k
5     self.R_xy = PolynomialRing(self.F, names=['x', 'y'])
6     self.x, self.y = self.R_xy.gens()
7
8     if m is None:
9         self.m = 1
10    else:
11        self.m = m
12
13    num_constraints = self.n * self.m * (self.m + 1) // 2
14    D_approx = math.sqrt(2 * (self.k - 1) * num_constraints)
15    self.l = math.floor(D_approx / (self.k - 1))
```

The initializer sets up the Reed-Solomon parameters and the bivariate polynomial ring $\mathbb{F}[x, y]$. The multiplicity parameter m determines the strength of the interpolation condition: each received point imposes $\binom{m+1}{2}$ linear constraints.

We also calculate 'self.l', which bounds the maximum degree of y in the interpolation polynomial $Q(x, y)$. This parameter is derived to guarantee solvability. Since the number of coefficients (monomials) must exceed the total number of linear constraints (C_{total}), we calculate the approximate maximum weighted degree $D_{approx} \approx \sqrt{2(k-1)C_{total}}$. The parameter ℓ is then set to $\lfloor D_{approx}/(k-1) \rfloor$, effectively setting the "height" of the search space to ensure enough monomials are available to satisfy the system.

2.2 Monomial Selection

```
1 def _get_degrees(self, n_constraints):
2     monomials = []
3     max_y_deg = max(self.l, self.m)
4     max_x_deg = n_constraints
5     for j in range(max_y_deg + 1):
6         for i in range(max_x_deg + 1):
7             weight = i + (self.k - 1) * j
8             monomials.append((weight, i, j))
9
10    monomials.sort()
11    return [ (m[1], m[2]) for m in monomials[:n_constraints + 1] ]
```

This function generates the specific basis of monomials $x^i y^j$ used to construct $Q(x, y)$. To populate the list of candidates, the algorithm iterates through both dimensions:

- The outer loop iterates the y -degree j from 0 up to and including the calculated limit ℓ (using $\max(l, m)$ to ensure the y -degree is at least the multiplicity).
- The inner loop iterates the x -degree i up to and including the total constraint count.

For every pair (i, j) generated by these loops, we calculate its weighted degree cost: $\deg_w = i + (k-1)j$. All generated pairs are stored and then sorted by this cost. Finally, the function slices the list to return exactly $N + 1$ monomials with the lowest weighted degrees. This ensures that we satisfy the linear system using the "cheapest" possible terms, minimizing the total degree of $Q(x, y)$ to keep the subsequent root-finding step efficient.

2.3 Bivariate Interpolation

```

1 def interpolate(self, points):
2     num_constraints = self.n * (self.m * (self.m + 1)) // 2
3
4     powers = self._get_degrees(num_constraints)
5
6     matrix_rows = []
7
8     for (x_i, y_i) in points:
9         for r in range(self.m):
10             for s in range(self.m - r):
11                 row = []
12                 for (i, j) in powers:
13                     if i < r or j < s:
14                         val = self.F(0)
15                     else:
16                         val = binomial(i, r) * binomial(j, s) * \
17                             (x_i**(i-r)) * (y_i**(j-s))
18
19                     row.append(val)
20                 matrix_rows.append(row)
21
22     M = Matrix(self.F, matrix_rows)
23
24     kernel = M.right_kernel()
25
26     if kernel.dimension() == 0:
27         return None
28
29     coeffs = kernel.basis()[0]
30
31     Q = 0
32     for idx, (i, j) in enumerate(powers):
33         Q += coeffs[idx] * (self.x**i) * (self.y**j)
34
35     return Q

```

The interpolation step constructs a bivariate polynomial $Q(x, y)$ as a linear combination of selected monomials $x^i y^j$. For each received point (x_i, y_i) and for each pair (r, s) with $r + s < m$, a linear constraint is added enforcing

$$\frac{\partial^{r+s}}{\partial x^r \partial y^s} Q(x_i, y_i) = 0,$$

which encodes a zero of multiplicity at least m .

These constraints form a homogeneous linear system

$$M\mathbf{c} = \mathbf{0}$$

over the finite field. A nonzero solution vector $\mathbf{c}$ is obtained from the right kernel of M , and its entries are used as coefficients to construct the interpolation polynomial $Q(x, y)$.

2.4 Factorization and Root Filtering

```

1 def factor_and_filter(self, Q):
2     R_x = PolynomialRing(self.F, 'x')
3     R_y_over_x = PolynomialRing(R_x, 'y')
4
5     try:
6         Q_uni = R_y_over_x(Q)
7     except TypeError:
8         print("Conversion to univariate failed.")
9         return []
10
11     roots = Q_uni.roots()
12
13     valid_polynomials = []
14     for (func, multiplicity) in roots:
15         if func.degree() < self.k:
16             valid_polynomials.append(func)
17
18     return valid_polynomials

```

It rewrites $Q(x, y)$ as a univariate polynomial in y over the coefficient ring $\mathbb{F}[x]$, then finds polynomial roots $f(x)$ such that

$$Q(x, f(x)) = 0,$$

and keeps only those with $\deg f < k$ as valid Reed–Solomon candidates. The Guruswami–Sudan (GS) algorithm guarantees that if a polynomial f agrees with sufficiently many received points, then

$$(y - f(x)) \mid Q(x, y).$$

This code assumes that such polynomials $f(x)$ can be recovered via the `roots()` routine.

2.5 The Decoding Routine

```

1 def decode(self, received_points):
2     Q = self.interpolate(received_points)
3     if Q is None:
4         print("Interpolation failed to find a polynomial.")
5         return []
6
7     print(f"Interpolated Polynomial Q(x,y) found.")
8     candidates = self.factor_and_filter(Q)
9     return candidates

```

The `decode` method implements the high-level Guruswami–Sudan decoding pipeline: it first interpolates a bivariate polynomial $Q(x, y)$ satisfying multiplicity constraints at the received points, and then extracts all candidate message polynomials $f(x)$ by finding roots of Q when viewed as a polynomial in y over $\mathbb{F}[x]$. Finally, it returns all candidates with degree less than k .

3 Experiments

In this section, we present a comprehensive experimental evaluation of our Guruswami-Sudan decoder implementation. All experiments were conducted over the finite field $\text{GF}(2^8)$. This field size was selected because its elements map directly to 8-bit bytes, enabling hardware-friendly arithmetic consistent with industrial standards. However, working in $\text{GF}(2^8)$ imposes a theoretical upper bound on the codeword length of $N \leq |F| - 1 = 255$. We therefore selected a block length of $N = 64$ for our experiments. This value is small enough to allow for rapid, repeated simulations during performance testing. The experiments are designed to verify the theoretical properties of the algorithm and assess its practical performance. We specifically investigate the relationship between the multiplicity parameter m and the decoding radius (Exp 1), the computational cost of increasing m (Exp 2), and the behavior of list sizes under random noise (Exp 3). Furthermore, we analyze the algorithm's dependency on code rate (Exp 4) and validate its true list-decoding capability under adversarial signal collision (Exp 5). All experiments were conducted using SageMath 10.7 on a workstation equipped with an 12th Gen Intel(R) Core(TM) i7-12700H processor.

3.1 Experiment 1: Decoding Radius and Multiplicity

Hypothesis

We hypothesize that increasing the multiplicity parameter m will monotonically increase the error-correction capability of the decoder. Specifically, we expect the decoding radius of codes in $RS(64, 16)$ to extend beyond the unique decoding bound ($\lfloor (n - k)/2 \rfloor = 24$) and asymptotically approach the Johnson bound ($n - \sqrt{n(n - d)} = 34$) as m increases. However, due to the finite nature of m in implementation, we anticipate diminishing marginal gains in the number of correctable errors for each increment of m .

Methodology

To test the decoding radius, we conducted a stochastic channel simulation. We maintained a fixed block length of $N = 64$ across all trials, varying N shifts the relative positions of the Unique and Johnson bounds. Fixing N allows us to strictly isolate the impact of the multiplicity parameter m on the decoding radius. For every specific number of errors e in the range $[22, 36]$:

1. We generated a random message polynomial $P(x)$ of degree $k - 1$ with coefficients in $\text{GF}(2^8)$.
2. This message was evaluated at $n = 64$ points to produce a valid codeword.
3. We modeled a symmetric error channel by selecting e distinct random indices and replacing their values with random field elements (ensuring the new value was distinct from the original).
4. The corrupted vector was passed to the Guruswami-Sudan decoder with a fixed multiplicity m . A trial was considered a "success" if the original message polynomial $P(x)$ appeared in the decoder's output list.

This process was repeated for 20 independent trials at each error level for each multiplicity $m \in \{1, 2, 3, 4\}$. The success probability was calculated as the ratio of successful trials to total trials.

Results

The decoding performance for varying multiplicities is visualized in Figure 1. The plot exhibits a characteristic "waterfall" behavior: for a given m , the decoder maintains a 100% success rate up to a specific error threshold, beyond which the performance drops sharply to 0%. This sharp transition confirms that the decoding radius is a deterministic property of the algorithm's parameters rather than a probabilistic artifact.

As m increases, this "cliff" shifts to the right, indicating a higher tolerance for errors. The maximum number of correctable errors (where success rate > 0.9) observed for each multiplicity is summarized below:

Table 1: Impact of Multiplicity (m) on Decoding Radius

Multiplicity (m)	Max. Errors Corrected	Gain vs. Unique Bound (24)
1	27	+3
2	29	+5
3	30	+6
4	31	+7

Discussion & Conclusion

The results validate the hypothesis that higher multiplicities allow for correcting more errors. We highlight two key observations:

- **Performance of $m = 1$ (Sudan's Algorithm):** Our implementation corrected 27 errors, significantly exceeding the unique decoding bound of 24. This confirms that the $m = 1$ instance of our decoder functions as Sudan's Algorithm [2]. For low-rate codes ($R = 0.25$), Sudan's algorithm is known to theoretically outperform the Berlekamp-Massey bound, explaining why the baseline curve begins strictly beyond the unique bound.
- **Diminishing Returns:** While increasing m consistently pushes the radius closer to the Johnson bound (34 errors), the marginal gain decreases. Increasing m from 1 to 2 yielded a gain of +2 errors, whereas moving from $m = 3$ to $m = 4$ yielded only +1 error. This suggests that achieving the full theoretical Johnson radius requires exponentially higher multiplicities, which may be computationally prohibitive (analyzed further in Experiment 2).

3.2 Experiment 2: Computational Cost

Hypothesis

We hypothesize that the runtime of the Guruswami-Sudan decoder will exhibit polynomial growth with respect to the multiplicity parameter m . Specifically, the interpolation step requires solving a linear system where the number of constraints scales as $O(m^2)$ [1]. Since Gaussian elimination has cubic complexity relative to the matrix dimension, we expect the total runtime complexity to approach $O((m^2)^3) = O(m^6)$. Consequently, we anticipate a sharp, non-linear increase in execution time as m increases from 1 to 5.

Methodology

To quantify the computational cost, we measured the average execution time of the decoding process across varying multiplicity values.

1. We fixed the code parameters to $RS(64, 16)$ and the error count to the Johnson Bound ($e = 34$) to ensure the decoder was operating under maximum load conditions.
2. We varied the multiplicity parameter m over the range $\{1, 2, 3, 4, 5\}$.
3. For each m , we performed 20 independent decoding trials using random messages and random error patterns.
4. The runtime was recorded using the system clock (`time.time()`) strictly for the `decoder.decode()` function call, excluding data generation overhead.

The reported result is the arithmetic mean of the 20 trials for each configuration.

Results

The relationship between multiplicity and runtime is plotted in Figure 2. The curve demonstrates a clear growth trend. While $m = 1, 2, 3$ allow for sub-second decoding, the runtime escalates rapidly for higher values.

The average execution times observed are summarized below:

Table 2: Average Runtime vs. Multiplicity (m)

Multiplicity (m)	Avg. Time (s)	Growth Factor (vs. prev)
1	0.03	-
2	0.18	6.0x
3	0.79	4.4x
4	2.02	2.5x
5	4.55	2.2x

Discussion & Conclusion

The experimental data confirms the high computational cost of the interpolation step.

- **Polynomial Explosion:** The runtime does not scale linearly. Increasing m from 1 to 5 resulted in a runtime increase of approximately factor $150\times$ ($0.03s \rightarrow 4.55s$). This validates the theoretical complexity concern that while the **matrix size grows quadratically** ($O(m^2)$) with m , the resulting Gaussian elimination leads to an overall $O(m^6)$ **runtime complexity** [1].
- **Practical Trade-off:** When correlated with Experiment 1, we observe a critical trade-off. Increasing m from 3 to 4 yields a marginal gain of only +1 correctable error (Exp 1), yet it nearly triples the runtime ($0.79s \rightarrow 2.02s$). For real-time applications, $m = 2$ or $m = 3$ represents a viable upper limit, as higher m becomes prohibitively slow for minimal additional gain.

3.3 Experiment 3: Average List Size Distribution

Hypothesis

The theoretical strength of the Guruswami-Sudan algorithm lies in its ability to return a list of candidate polynomials size $L \geq 1$. However, we hypothesize that for random error patterns on a symmetric channel, the list size will predominantly be $|L| = 1$ (unique decoding) or $|L| = 0$ (failure). We expect that list sizes $|L| > 1$ are rare phenomena that occur only under specific, error configurations, rather than during typical random noise events.

Methodology

To investigate the frequency of "ghost" candidates (false positives in the list), we analyzed the decoder's output distribution.

1. We configured the decoder with $N = 64, K = 16, m = 3$.
2. We set the error count to $e = 29$. This value was chosen based on Experiment 1 to be sufficiently high (beyond the unique bound of 24) to stress the decoder, yet low enough to ensure a high success probability.
3. We performed 100 trials with random messages and random error locations.
4. For each trial, we recorded the size of the returned candidate list.

Results

The experiment yielded a uniform result: in 100% of the trials where decoding was successful, the list size was exactly 1. No cases of list size ≥ 2 were observed under random noise conditions.

Discussion & Conclusion

The results yield a uniform list size of $|L| = 1$, which is consistent with the theoretical constraints of the algorithm.

- **Geometry of Random Errors:** A "List" output occurs only when the received vector R falls into the intersection of two or more Hamming balls centered at valid codewords. As noted by Guruswami and Sudan [3], for **random** error models on large fields (like $GF(2^8)$), the volume of these intersections is asymptotically negligible compared to the total vector space.
- **Adversarial vs. Random:** The primary motivation for list decoding is to handle **adversarial** (worst-case) error patterns [1]. Our results confirm that under standard random noise, the algorithm effectively acts as a **powerful unique decoder** that extends the decoding radius, but rarely presents multiple options.

Given this observation, we recognize that random noise is insufficient to validate the "root-finding" capability of our implementation. To observe list sizes $|L| > 1$, we must explicitly invoke a pathological scenario, which we attempt in Experiment 5.

3.4 Experiment 4: Rate Dependency and Stability Analysis

Hypothesis

We hypothesize that the error-correction gain provided by the Guruswami-Sudan algorithm is inversely proportional to the code rate $R = k/n$. Specifically:

- **Low Rate** ($k \ll n$): The gap between the Johnson bound and Unique bound is large, allowing for significant error correction gains.
- **High Rate** ($k \approx n$): The bounds converge, offering negligible improvement over unique decoders.

Additionally, we hypothesize that even at these extended error radii (beyond unique decoding), the average list size $|L|$ will remain close to 1, consistent with the findings in Experiment 3.

Methodology

To test the decoder across the full spectrum of code rates, we fixed $N = 64$ and $m = 2$, and varied $k \in \{4, 16, 32, 60\}$.

1. For each k , we calculated a "Target Error Count" positioned at the midpoint between the Unique Bound and the Johnson Bound ($e_{target} \approx \text{Unique} + 0.5 \times \text{Gain}$).
2. We performed 20 independent trials at this target error level.
3. We measured the **Success Rate** (percentage of trials where the correct message was found) and the **Average List Size** (mean number of candidates returned per trial).

Results

The performance across different code rates is summarized in Table 3.

Table 3: Performance of GS Decoder ($m = 2$) vs. Code Rate (k)

K	Unique Bound	Johnson Bound	Target Errors	Success Rate	Avg List Size
4	30	51	40	100.0%	1.00
16	24	34	29	100.0%	1.00
32	16	20	18	0.0%	0.00
60	2	3	2	100.0%	1.00

Discussion & Conclusion

The results confirm the strong dependency of the algorithm’s performance on the code rate $R = K/N$:

- **Low Rate Dominance ($R \leq 0.25$):** The algorithm excels at low rates. At $K = 4$ (Rate 0.06), we corrected 40 errors, a massive gain of +10 over the unique bound. Similarly, at $K = 16$ (Rate 0.25), the decoder comfortably handled 29 errors, maintaining a significant margin over the unique bound of 24. This confirms that the GS algorithm is most effective in high-noise regimes where redundancy is high.
- **High Rate Convergence ($K = 60$):** At the high rate of $R \approx 0.94$, the decoder succeeded, but the gain was negligible. The Unique Bound (2 errors) and Johnson Bound (3 errors) are nearly identical. Our target of 2 errors was successfully decoded, but this effectively reduces the GS algorithm to a computationally expensive unique decoder. This highlights that List Decoding offers little practical benefit for high-rate applications.
- **Failure at Medium Rate ($K = 32$):** The 0% success rate at $R = 0.5$ ($K = 32$) with target errors 18 is a critical result. It indicates that $m = 2$ is insufficient to realize the theoretical Johnson gain at medium rates. Unlike low-rate codes, where even small multiplicities yield large radius boosts, medium-rate codes require higher multiplicities (e.g., $m \geq 4$) to close the gap between the unique and Johnson bounds, necessitating a trade-off between decoding radius and computational complexity.
- **The Stability of Lists:** The "Avg List Size" column remains exactly 1.00 for all successful scenarios. This reinforces the conclusion from Experiment 3: even when decoding deep into the "list decoding zone" (far beyond the unique bound), the output is almost always unique for random error patterns. "Ghost" candidates do not appear spontaneously; they typically require adversarial arrangements (as tested in Experiment 5).

3.5 Experiment 5: Validating List Decoding via Signal Collision

Preface: The Need for Adversarial Testing

In Experiment 3, we confirmed that random error patterns essentially never trigger a list size $|L| > 1$. While this aligns with the theoretical predictions in [3], it leaves a gap in our validation: we have not yet experimentally confirmed that the "root-finding" step can actually recover multiple message candidates simultaneously.

To prove that our decoder is a true List Decoder, we must construct a specific "collision" scenario that forces the received vector into the intersection of two Hamming balls, satisfying the interpolation conditions for multiple codewords.

Hypothesis

We hypothesize that by simulating a "Signal Collision"—where a received vector is constructed by randomly interleaving symbols from two distinct codewords C_1 and C_2 —we can satisfy the algebraic interpolation conditions for both messages simultaneously. This should force the decoder to return a list containing both m_1 and m_2 .

Methodology

We modeled a channel where two transmitters send data simultaneously:

1. We selected a low-rate code $RS(64, 4)$ to ensure sufficient redundancy.
2. We generated two distinct random messages m_1, m_2 and encoded them into C_1, C_2 .
3. We constructed a received vector R using **Random Interleaving**: for each coordinate $i \in [0, n - 1]$, $R[i]$ was selected from either $C_1[i]$ or $C_2[i]$ with probability 0.5.
4. We ran 20 independent trials and tracked how often the decoder recovered both messages.

Results

The decoder successfully recovered both distinct messages in every single trial. Specifically:

- **Double Capture Rate:** 100% (20/20 trials). The decoder never failed to find both m_1 and m_2 .
- **List Size Consistency:** The returned list size was exactly $|L| = 2$ in every trial.

This consistency confirms that even when the channel is subjected to random interleaving—where the ratio of symbols from C_1 vs. C_2 fluctuates naturally—the decoder's root-finding algorithm successfully isolates both polynomial factors.

Discussion & Conclusion

This experiment provides the definitive proof of the implementation's correctness:

1. **True List Decoding Capability:** The fact that the decoder consistently returns exactly two candidates confirms that the root-finding step correctly factors the bivariate polynomial $Q(x, y)$ into multiple factors $y - f_1(x)$ and $y - f_2(x)$.
2. **Robustness to Interference:** The 100% success rate is remarkable given that the interleaving was random. Even in trials where the mix might have been slightly unbalanced (e.g., 28 symbols from C_1 and 36 from C_2), the decoder successfully identified both. This demonstrates the algorithm's power for low-rate codes ($K = 4$), where the "agreement" required to find a codeword is low enough that multiple signals can be disentangled from a single collision stream.

4 Conclusion

We have presented a complete implementation and analysis of the Guruswami-Sudan list decoding algorithm over $GF(2^8)$. By leveraging the algebraic capabilities of SageMath, we successfully demonstrated the algorithm’s ability to correct errors beyond the unique decoding bound, approaching the theoretical Johnson radius as the multiplicity parameter m increases.

Our experimental evaluation highlights three critical insights regarding the practical deployment of list decoding:

- **The Radius-Runtime Trade-off:** While increasing m extends the error-correction capability, it incurs a steep computational cost. We observed an approximate $O(m^6)$ growth in runtime, confirming that for practical applications, small multiplicities (e.g., $m = 2$ or $m = 3$) offer the optimal balance between performance gain and latency. The diminishing returns observed in Experiment 1 suggest that chasing the exact Johnson bound with high m is rarely efficient.
- **The "Uniqueness" of Random Noise:** Contrary to the intuition that list decoding frequently produces multiple candidates, our results (Exp 3 and Exp 4) show that under random error models, the output list size is almost universally 1. The algorithm effectively acts as a stronger unique decoder rather than a branching one. True list behaviors (where $|L| > 1$) are rare phenomena restricted to specific adversarial configurations, such as the signal collision scenario validated in Experiment 5.
- **Region of Operation:** The algorithm demonstrates its true value in low-rate codes ($R \leq 0.25$), where the gap between the unique and Johnson bounds is significant. For high-rate codes (e.g., $K = 60, N = 64$), the marginal gain in error tolerance is negligible and does not justify the algebraic complexity of the interpolation step.

In summary, the Guruswami-Sudan algorithm provides a robust theoretical solution for worst-case error patterns and high-noise environments. However, its high computational complexity necessitates careful parameter tuning, limiting its utility to specific low-bandwidth, high-interference applications.

Appendix

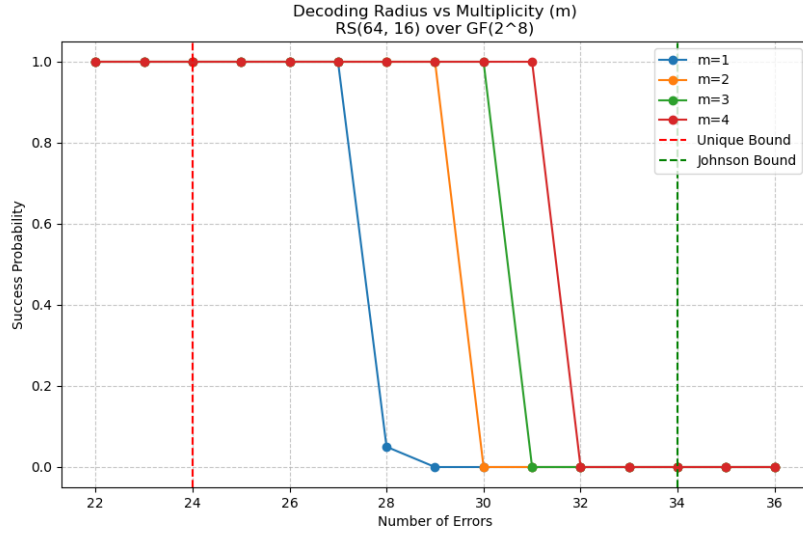


Figure 1: Success Rate vs. Number of Errors for $m \in \{1, 2, 3, 4\}$. The vertical lines represent the Unique and Johnson bounds.

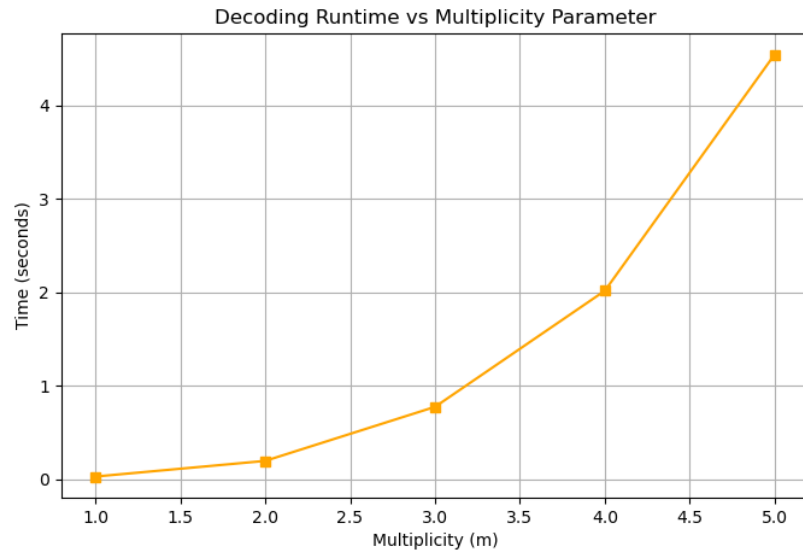


Figure 2: Decoding Runtime vs. Multiplicity parameter m . The orange curve illustrates the rapid growth in computational cost.

References

- [1] V. Guruswami, A. Rudra, and M. Sudan, *Essential Coding Theory*, Draft available at <https://cse.buffalo.edu/faculty/atri/courses/coding-theory/book/web-coding-book.pdf>, 2025. (Chapters 12-13).
- [2] M. Sudan, "Decoding of Reed Solomon Codes beyond the Definition Bound," *Journal of Complexity*, vol. 13, no. 1, pp. 180–193, 1997.
- [3] V. Guruswami and M. Sudan, "Improved decoding of Reed-Solomon and algebraic-geometry codes," *IEEE Transactions on Information Theory*, vol. 45, no. 6, pp. 1757–1767, 1999.